An MARL-based Task Scheduling Algorithm for Cooperative Computation in a multi-UAV Edge Computing System

Luyinru Yang, Jun Zheng, and Yuan Zhang
School of Information Science and Engineering, Southeast University, Nanjing, China
Email: junzheng@seu.edu.cn

***Abstract*—This paper investigates the task scheduling problem for cooperative computation in a multi-UAV edge computing system. The problem is formulated as an integer nonlinear programming problem with an objective to maximize the average number of computation tasks successfully processed in each time slot. To solve the formulated problem using a reinforcement learning method, the problem is first reformulated as a decentralized partially observable Markov decision process (Dec-POMDP) and transformed into an average-reward maximization problem. Then, a multi-agent reinforcement learning (MARL)-based task scheduling algorithm (MTS) is proposed to solve the transformed problem. The proposed MTS algorithm introduces a QMIX-based training framework, in which the local policies of all agents are jointly trained by a central controller in a centralized manner and each agent executes actions independently according to its local observation and local policy in a decentralized manner. Moreover, recurrent neural networks (RNNs) are also introduced to deal with the partial observability of POMDP. Using the proposed MTS algorithm, each UAV is able to make optimal scheduling decisions independently to enable efficient cooperative computation between UAVs. Simulation results demonstrate that the proposed MTS algorithm can achieve better performance than benchmark algorithms in terms of the number of tasks successfully processed in the system.**

***Index Terms*—UAV; MARL; task scheduling; edge computing; cooperative computation**

I. Introduction

With the characteristics of convenient deployment, high flexibility and low cost, unmanned aerial vehicles (UAVs) have been recognized as a promising technology for executing a variety of military and civilian tasks in the air [1, 2]. A UAV can be used as an aerial station to provide temporary computing services to ground users in an emergency situation, remote area, or hotspot [3, 4]. However, the computation resources of a single UAV are limited, which would affect the quality of services provisioned by a single UAV. To deal with the limitation, multiple UAVs can be deployed over a service area as an aerial edge computing platform to cooperatively execute a common task and thus provide better services for ground users [5]. In such an application scenario, the service load of each UAV could be different and dynamically change. Multiple UAVs need to cooperatively schedule their tasks among each other in a distributed manner in order to fully exploit their communication and computation resources. Therefore, task scheduling becomes a critical issue in a multi-UAV edge computing system and an efficient task scheduling algorithm is required to address this issue. To this end, many existing studies formulate a task scheduling problem as an optimization problem and use conventional optimization methods to solve the problem. However, most of those formulated task scheduling problems are usually NP-hard and are computationally difficult to tackle by using conventional model-based methods. Moreover, conventional model-based methods require prior knowledge of environment information and thus are hardly able to deal with complicated dynamic scenarios with unknown environment information. In contrast, model-free reinforcement learning (RL) methods can learn a decision policy from interactions with an environment through trials and errors. For an RL method, prior knowledge of environment information is not essential for its learning process. Moreover, an RL method can quickly respond to a dynamic environment and make decisions as it has a low computational complexity in generating actions. This motivated us to use a model-free RL method to solve the task scheduling problem.

In this paper, we investigate the task scheduling problem for cooperative computation in a multi-UAV MEC system and introduces an RL method to deal with the problem. The main contributions of this work can be summarized as follows:

1) The task scheduling problem under investigation is formulated as an integer nonlinear programming problem with the objective to maximize the average number of computation tasks successfully processed in each time slot.
2) The formulated problem is first reformulated as a decentralized partially observable Markov decision process (Dec-POMDP) and further transformed into an average-reward maximization problem. A multi-agent reinforcement learning (MARL)-based task scheduling algorithm (MTS) is then proposed to solve the transformed problem.
3) The proposed MTS algorithm introduces a QMIX-based centralized training and decentralized execution (CTDE) framework to enable each UAV to make scheduling decisions independently. Recurrent neural networks (RNNs) are introduced to deal with the partial observability

Manuscript received xx XXX 2025; revised xx XXX 2025; accepted xx XXX 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62471130 and Grant 62271134, and in part by the National Key R&D Program of China under Grant 2023 YFB4503302. The review of this article was coordinated by Dr. Qing Yang. (Corresponding author: Jun Zheng)

Luyinru Yang and Yuan Zhang are with the School of Information Science and Engineering, Southeast University, Nanjing 210096, China (e-mail: lyryang@seu.edu.cn, y.zhang@seu.edu.cn).

Jun Zheng is with the Purple Mountain Laboratories, Nanjing 211111, China, and the School of Information Science and Engineering, Southeast University, Nanjing 210096, China (e-mail: junzheng@seu.edu.cn).

of POMDP and a new network structure with an RNN combined with a deep Q-network (DQN) is designed for an agent network to enable an agent to make multiple decisions at a step.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the considered system model and formulates the optimization problem. Section IV presents the proposed MTS algorithm. Section V evaluates the performance of the proposed algorithm through simulation results. Section VI concludes this paper.

II. Related Work

Computation offloading and task scheduling in multi-UAV edge computing systems have been widely studied in the literature. Luan et al. [6] formulated the topology reconstruction and subtask scheduling optimization problem in a UAV-assisted MEC emergency network with an objective to minimize the average completion time of dependent subtasks and proposed a hierarchical hybrid subtask scheduling algorithm to solve the problem. Qin et al. [7] studied a joint task scheduling, resource allocation and UAV trajectory optimization problem for air-ground collaborative MEC and proposed an iterative algorithm to minimize a weighted age of information (AoI) of all terrestrial users. Li et al. [8] studied a joint computation offloading and resource allocation problem in an air-ground integrated vehicular edge computing network in order to minimize the overall task offloading delay. Based on convex optimization methods and game theory, several optimization algorithms are proposed to solve the optimization problem. Kuang et al. [9] formulated the task offloading, resource allocation, and UAV deployment problem in a multi-UAV-enabled MEC system to maximize the number of tasks offloaded to UAVs and proposed a two-layer optimization method based on differential evolution and convex optimization to solve the formulated nonconvex nonlinear optimization problem. Tun et al. [10] formulated the computation offloading and resource allocation optimization problem in a collaborative multi-UAV-assisted MEC system with an objective to minimize the total latency experienced by users. The problem is decomposed into three subproblems and solved using the Lagrangian relaxation method and the alternating direction method of multipliers.

The above studies mainly use conventional model-based methods such as convex optimization to solve the optimization problems. However, these methods require prior knowledge of environment information and thus are hardly able to deal with complicated dynamic scenarios with unknown environment information. In the context of model-free RL methods, most of existing related studies focus on task offloading from ground users to UAV nodes without considering cooperative computation between UAV nodes, which may not be able to fully exploit the computation resources of the UAVs. For example, Chen et al. [11] investigated an air-ground integrated MEC system and formulated a non-cooperative stochastic game to model task offloading of mobile users with an objective to maximize each user's expected payoff in terms of information freshness and energy consumption. The stochastic game is transformed into a single-agent Markov decision process (MDP) and an online deep RL scheme is proposed to optimize users' decisions on channel auction, computation offloading and packet scheduling. Kang et al. [12] considered the resource allocation and task offloading problem in a hierarchical aerial computing system, and proposed a multiagent proximal policy optimization-based algorithm to maximize the number of computed tasks in the system. Yu et al. [13] proposed a QMIX-based MARL algorithm to solve the joint computation offloading and UAV trajectory optimization problem in a heterogeneous UAV-swarm-enabled edge computing network to minimize the sum of the maximum end-to-end delay in the system. Luo et al. [14] considered an edge computing enabled multi-UAV cooperative target search problem and proposed a DQN-based strategy to minimize the uncertainty of a search area by jointly optimizing task offloading and flying orientation of UAVs. Xu et al. [15] considered a multi-UAV cooperative computation framework where a UAV can offload its tasks to nearby UAVs with idle computation resources. A long-term optimization problem is formulated to minimize the total task completion time and the total system energy consumption by jointly optimizing the inter-UAV task offloading ratio, UAV trajectory and resource allocation. The proximal policy optimization (PPO) is introduced in a two-layer loop structure to solve the problem. Li et al. [16] studied a joint computation offloading and trajectory planning optimization problem in a multi-UAV-enabled MEC network and proposed an RL algorithm to solve the problem, in which expert knowledge is used to assist agent learning. Considering the current research status in this area, further efforts are still needed to explore model-free RL methods in task scheduling between multiple UAVs for cooperative computation.

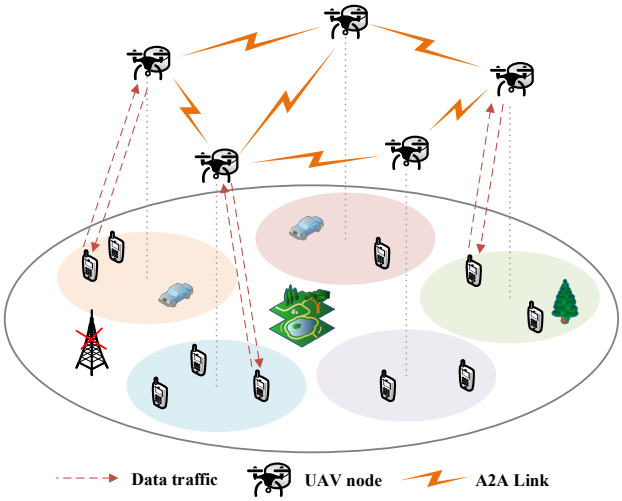

Fig. 1. System model.

III. System Model and Problem Formulation

In this section, we first describe the system model and then formulate the task scheduling optimization problem considered in this paper.

A. System Model

We consider a multi-UAV edge computing system, where multiple UAVs are deployed over a geographical area as aerial base stations to provide edge computing services for ground users, as shown in Fig. 1. The geographical area covers several service areas. Each UAV is deployed over a service area where

ground users are randomly distributed. Each user generates computation tasks randomly and transmits generated tasks to its local UAV for processing. All the UAVs provide computing services for all the ground users in a cooperative manner. Each UAV can either process a local task by itself or offload the task to a neighboring UAV depending on the load conditions of the UAVs in the system. Each UAV uses a different frequency band to communicate with ground users in its service area, where each ground user is allocated an orthogonal channel on the band for task offloading. Meanwhile, all UAVs use another frequency band to communicate with each other and each couple of neighboring UAVs are allocated an orthogonal channel on the band for task offloading between each other. Thus, there is no interference on the channels either between UAVs or between ground users and their local UAVs in the system.

The overall system provides computing services for the ground users in a time-slotted manner. Each time slot has fixed length Δt and its index is denoted by $t \in \mathcal{T} = \{1,2,\ldots,T\}$. Let $\mathcal{U} = \{U_n, n = 1,\ldots,N\}$ denote a set of UAV nodes and $\mathcal{G} = \{G_m, m = 1,\ldots,M\}$ denote a set of ground users. The position of UAV node $U_n \in \mathcal{U}$ is denoted by $\boldsymbol{pos}^n = (x^n, y^n, h^n)$ and the position of ground user $G_m \in \mathcal{G}$ is denoted by $\boldsymbol{pos}^m = (x^m, y^m, 0)$. Thus, the distance between UAV node U_n and ground user G_m is

$$dis^{mn} = \sqrt{(x^n - x^m)^2 + (y^n - y^m)^2 + (h^n)^2}, \quad (1)$$

and the distance between UAV node U_n and UAV node $U_{n'}$ is

$$dis^{nn'} = \sqrt{(x^n - x^{n'})^2 + (y^n - y^{n'})^2 + (h^n - h^{n'})^2}. \quad (2)$$

B. Channel Model

1) Air to Ground (A2G) Channel

The channel between a UAV and a ground user is an A2G channel, which could be either LoS or Non-Line-of-Sight (NLoS) based on the probabilistic LoS channel model [17]. According to [17], the LoS path loss and NLoS path loss from ground user G_m to UAV node U_n are respectively given by

$$PL_{LoS}^{mn} = 20\log\frac{4\pi f^m}{c} + 20\log dis^{mn} + \eta_{LoS}, \quad (3)$$

$$PL_{NLoS}^{mn} = 20\log\frac{4\pi f^m}{c} + 20\log dis^{mn} + \eta_{NLoS}, \quad (4)$$

where η_{LoS} and η_{NLoS} denote the additional attenuation factors in LoS and NLoS conditions respectively, and f^m is the carrier frequency of ground user G_m. The probability that the channel is a LoS channel is given by

$$p_{LoS}^{mn} = \frac{1}{1 + a\exp(-b(\theta^{mn} - a))}, \quad (5)$$

where a and b are constants which depend on the environment and $\theta^{mn} = (180/\pi)\cdot\sin^{-1}(h^n/dis^{mn})$ is the elevation angle between ground user G_m and UAV node U_n. Accordingly, the mean path loss of the A2G channel is given by

$$\overline{PL}^{mn} = p_{LoS}^{mn} PL_{LoS}^{mn} + (1 - p_{LoS}^{mn}) PL_{NLoS}^{mn}. \quad (6)$$

Thus, the transmission rate of the channel between ground user G_m and UAV node U_n is given by

$$R^{mn} = B^G \log_2\left(1 + \frac{P^G}{10^{\overline{PL}^{mn}/10} N_0 B^G}\right), \quad (7)$$

where B^G is the bandwidth of the A2G channel, P^G is the transmission power of a ground user, and N_0 is the power spectral density of the noise on the channel.

2) Air to Air (A2A) Channel:

The channel between two UAV nodes is an A2A channel, where LoS is dominant. For UAV node U_n, its neighboring UAV nodes can be represented as $\mathcal{U}_{neigh}^n = \{U_{n'} \mid 0 < dis^{nn'} < dis^{\max}, \forall U_{n'} \in \mathcal{U}\}$, where $dis^{\max}$ is the communication distance of a UAV node. Thus, the path loss of the channel can be described using a free space path loss model [17]:

$$PL^{nn'} = 20\log\frac{4\pi f^{nn'}}{c} + 20\log dis^{nn'}, \quad (8)$$

where $f^{nn'}$ is the carrier frequency of the channel between UAV node U_n and UAV node $U_{n'}$. Thus, the transmission rate of the channel from UAV node U_n to UAV node $U_{n'}$ is given by

$$R^{nn'} = B^U \log_2\left(1 + \frac{P^U}{10^{\overline{PL}^{nn'}/10} N_0 B^U}\right), \quad (9)$$

where B^U is the bandwidth of the A2A channel and P^U is the transmission power of a UAV node.

C. Computation Model

As assumed earlier, computation tasks are randomly generated by the ground users. Assume that each ground user in the system generates a computation task at a generation rate p_{gen} in each time slot. Each task is represented by a four-tuple $task\_k = (d^k, c^k, t^k, \tau_{\max}^k)$, where $k \in \mathcal{K} = \{1,2,\ldots\}$ is a task index, d^k denotes the data size of $task\_k$ in bit, c^k denotes the number of CPU cycles required to process one bit data, t^k denotes the index of the time slot when $task\_k$ is generated, $\tau_{\max}^k$ denotes the maximum tolerant delay of $task\_k$. A UAV can process only one computation task at a time and sequentially process computation tasks according to a first-in-first-out (FIFO) principle. Thus, for $task\_k$, the computation delay is given by

$$\tau_{comp}^k = \frac{d^k c^k}{f^{CPU}}, \quad (10)$$

where f^{CPU} denotes the CPU frequency of each UAV.

D. Service Delay

In the considered system, once a ground user generates a computation task, it will transmit the task to its local UAV. Each UAV may receive one or more tasks in each time slot, which are either generated by local users or offloaded from

other UAVs. In each time slot, a UAV node will make scheduling decisions on the computation tasks arriving in the last time slot in order of arrival time. After a task arrives at a UAV node, it will first be buffered at the node waiting for the UAV node to make a scheduling decision. In this case, the UAV node is called the hosting UAV node of the task. For a particular arrival task, its hosting UAV node will decide whether to process the task by itself or offload the task to another UAV for processing based on its own load state and the load states of its neighboring UAV nodes. The load state of a UAV node mainly depends on the states of the task queues maintained at the UAV node. Each UAV maintains three task queues: an arrival task queue, a computation task queue, and an offloading task queue. The arrival task queue is used to buffer tasks arriving in the last time slot. The computation task queue is used to buffer tasks waiting to be processed in the current time slot. The offloading task queue is used to buffer tasks waiting to be offloaded to other UAV nodes in the current time slot. Assume that each UAV broadcasts the state information of its own task queues to its neighboring UAV nodes in each time slot. Thus, when UAV node U_n makes a scheduling decision, the task queue states of its neighboring UAV nodes in the last time slot are known to UAV node U_n while those in the current slot are unknown due to the transmission delay. After a decision is made for a task, the task will be moved to either the computation task queue or the offloading task queue waiting for subsequent processing, depending on the decision.

The service delay of $task\_k$ is defined as the time from the instant when $task\_k$ is generated to the instant when the processing of $task\_k$ is completed, which is comprised of a transmission delay, a queuing delay and a computation delay and can be expressed as

$$\tau_{sev}^k = \tau_{tran}^k + \tau_{que}^k + \tau_{comp}^k , \tag{11}$$

where τ_{tran}^k denotes the transmission delay, τ_{que}^k denotes the queuing delay, and τ_{comp}^k denotes the computation delay. For the transmission delay τ_{tran}^k, it is comprised of $task\_k$'s transmission delay from ground user G_m to UAV node U_n and that from UAV node U_n to UAV node $U_{n'}$, which are respectively given by $\tau_{tran,mn}^k = \frac{d^k}{R^{mn}}$, $\tau_{tran,nn'}^k = \frac{d^k}{R^{nn'}}$. (12-13)

For the computation delay τ_{comp}^k, it is given by Eq. (10). For the queuing delay τ_{que}^k, it is comprised of $task\_k$'s waiting time in a computation task queue and that in an offloading task queue, which depend on the states of the queues at $task\_k$'s hosting UAV node and are unnecessarily derived in the context of this paper.

E. Problem Formulation

We consider the task scheduling optimization problem in the multi-UAV edge computing system shown in Fig. 1. Let $x_t^k \in \mathcal{U}$ denote the scheduling decision made by a hosting UAV or a ground user in time slot t for $task\_k$, which is the target offloading UAV node selected in time slot t for $task\_k$. Let $y_t^k \in \{0\} \bigcup \mathcal{U}$ denote the hosting node of $task\_k$ in time slot t, where $y_t^k = 0$ indicates the task is hosted by a ground user and $y_t^k \in \mathcal{U}$ indicates the task is hosted by a UAV node. Note that y_t^k is not a decision variable and it can be obtained from the system directly.

According to the values of (x_t^k, y_t^k), the statuses of a computation task have three cases, i.e.,

1) $x_t^k = U_n, y_t^k = 0$: $task\_k$ is generated at a ground user and has not been offloaded to UAV node U_n;

2) $x_t^k = U_n, y_t^k = U_n$: $task\_k$ is hosted at UAV node U_n and is scheduled to be processed by UAV node U_n itself;

3) $x_t^k = U_{n'}, y_t^k = U_n (n' \neq n)$: $task\_k$ is hosted at UAV node U_n but is scheduled to be processed by UAV node $U_{n'}$.

Let $\boldsymbol{q}_{n,t} = (\boldsymbol{q}_{n,t}^a, \boldsymbol{q}_{n,t}^c, \boldsymbol{q}_{n,t}^o)$ denote a set of task queues maintained at UAV node U_n in time slot t, where $\boldsymbol{q}_{n,t}^a = \{task\_k \mid k \in \mathcal{K}, x_t^k = U_n, y_{t-1}^k \neq U_n\}$ denotes the state of the arrival task queue in time slot t, which buffers the tasks arriving in time slot t-1, $\boldsymbol{q}_{n,t}^c = \{task\_k \mid k \in \mathcal{K}, x_t^k = U_n, y_{t-1}^k = U_n\}$ denotes the state of the computation task queue at the beginning of time slot t, which buffers the tasks waiting to be locally processed, and $\boldsymbol{q}_{n,t}^o = \{task\_k \mid k \in \mathcal{K}, x_t^k \neq U_n, y_t^k = U_n\}$ denotes the state of the offloading task queue at the beginning of time slot t, which buffers the tasks waiting to be offloaded to other UAV nodes. Assume that each UAV broadcasts the state information of its own task queues to its neighboring UAV nodes in each time slot. Also, let $\boldsymbol{\Pi}_t^n = \{x_t^k \mid task\_k \in \boldsymbol{q}_{n,t}^a\}$ denote a set of scheduling decisions made at UAV node U_n in time slot t and $\boldsymbol{\Pi} = \{\boldsymbol{\Pi}_t^n \mid \forall U_n \in \mathcal{U}, t \in \mathcal{T}\}$. Therefore, the objective of the task scheduling optimization problem considered in this paper is to make an optimal scheduling decision for each task generated by the ground users in each time slot such that the average number of computation tasks successfully processed in each time slot is maximized, i.e.,

P1: Objective $$\max_{\boldsymbol{\Pi}} \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} K_t^{succ} , \tag{14a}$$

subject to

$$x_t^k \in \{U_n\} \bigcup \mathcal{U}_{neigh}^n, \forall task\_k \in \{task\_k \mid k \in \mathcal{K}, y_t^k = U_n\}, t \in \mathcal{T} , \tag{14b}$$

$$\tau_{sev}^k \leq \tau_{\max}^k, \forall task\_k \in \{task\_k \mid k \in \mathcal{K}, 0 < t^k < T\} , \tag{14c}$$

where K_t^{succ} denotes the number of computation tasks successfully processed in time slot t, (14b) specifies that a task hosted at UAV node U_n can only be scheduled to be processed at the hosting UAV node or one of UAV node U_n's neighbors, and (14c) specifies that the service delay of a task should not exceed the maximum tolerant delay of a task.

The above formulated problem is an integer nonlinear programming problem, which is NP-hard, and it is difficult to find an optimal solution to the problem using a conventional optimization method. Thus, we will attempt to introduce RL and propose an MARL-based task scheduling (MTS) algorithm to solve the problem.

IV. MARL-BASED TASK SCHEDULING ALGORITHM

This section presents the proposed MTS algorithm. An RL method is usually used to solve an optimization problem which is modeled as a Markov Decision Process (MDP), in which an agent takes an action with the objective to maximize its reward. An MARL method is an RL method which is used to solve a problem where multiple agents exist in a system. To use an MARL method to solve the formulated problem, the problem needs to be reformulated as an MDP first and the objective of the MDP should be equivalent to that of the original optimization problem. In the considered system, each UAV node makes a task scheduling decision based on its local observation in a decentralized manner. Since a UAV is unable to know the state information on all other UAVs, i.e., a global state of the system is only partially observed by each UAV, the task scheduling optimization problem under consideration can be reformulated as a decentralized partially observable Markov decision process (Dec-POMDP). Based on the Dec-POMDP model, the original optimization problem is further transformed into an average-reward maximization problem. The proposed MTS algorithm is intended to solve the transformed problem.

A. Problem Transformation

(1) Dec-POMDP modeling

A Dec-POMDP can be characterized by a tuple $\Gamma = (\mathcal{N}, \mathcal{S}, \{\mathcal{A}^n\}_{n=1}^N, \mathcal{P}, \mathcal{R}, \{\mathcal{O}\}_{n=1}^N)$, where $\mathcal{N}$ denotes a set of agents, $\mathcal{S}$ denotes a global state space of an environment, $\{\mathcal{A}^n\}_{n=1}^N$ denotes a set of action spaces of the agents, $\mathcal{P}$ denotes a state transition probability function, $\mathcal{R}$ denotes an immediate reward function, and $\{\mathcal{O}^n\}_{n=1}^N$ denotes a set of observation spaces of the agents. To solve problem P1, the multi-UAV edge computing system is analogous to the environment and a UAV is analogous to an agent in a Dec-POMDP. The other elements in a Dec-POMDP are defined below.

- State:

A state of an environment is a set of information on all elements required for optimization in the environment. For the considered system, a state includes the information on all computation tasks in the system in a time slot. Let s_t denote the state of the system in time slot *t*. Obviously, we have $s_t \in \mathcal{S}$.

- Observation:

An observation is a set of environmental states or environmental information observed by an agent. For the considered system, the observation of UAV node U_n includes the states of arriving tasks in the current time slot, the states of UAV node U_n's task queues in the current time slot, and the states of the task queues of UAV node U_n's neighboring nodes in the last time slot.

Let o_t^n denote the observation of UAV node U_n in time slot *t* and $\boldsymbol{a}_t^n$ denote a set of actions executed by UAV node U_n in time slot *t*. Thus, we have $\boldsymbol{o}_t^n = (\{task\_k\}_{task\_k \in \boldsymbol{q}_{n,t}^a}, \boldsymbol{q}_{n,t}, \{\boldsymbol{q}_{n',t-1}\}_{U_{n'} \in \mathcal{U}_{neigh}^n})$. Considering the environment information is partially observable and imperfect, the observation-action history $\boldsymbol{\tau}_t^n = \{o_1^n, \boldsymbol{a}_1^n, o_2^n, \boldsymbol{a}_2^n, \ldots, o_t^n\}$, instead of individual o_t^n, is usually used during the process of solving the POMDP.

- Action:

An action is executed by an agent to interact with the environment. For the considered system, an action is executed by a UAV node to make a scheduling decision for an arriving task in a time slot. In each time slot, a UAV node needs to choose an action from an action space and execute the action for each task arriving in the last time slot. The number of actions executed by a UAV node in the current time slot depends on the number of tasks arriving in the last time slot. A set of actions executed by UAV node U_n in time slot *t* is denoted by $\boldsymbol{a}_t^n = \{x_t^k \mid task\_k \in \boldsymbol{q}_{n,t}^a\}$, which is actually a set of scheduling decisions made at UAV node U_n in time slot *t*, i.e., $\boldsymbol{\Pi}_t^n = \{x_t^k \mid task\_k \in \boldsymbol{q}_{n,t}^a\}$. The action space of a UAV node depends on a set of neighboring UAVs within the communication distance of the UAV. All possible actions of all UAV nodes constitute a joint action space, i.e., $\{\mathcal{A}^n\}_{n=1}^N$. Let $\boldsymbol{a}_t$ denote a joint action in time slot *t*.

- State transition probability function:

The state transition probability function $\mathcal{P}$ determines the probability that the state of the environment transits from the current state to a new state after an action is executed. For the considered system, the state of the system will transit from the current state s_t in time slot *t* to a new state s_{t+1} in time slot *t*+1 with a state transition probability $\mathcal{P}(s_{t+1} \mid s_t, \boldsymbol{a}_t)$ after action $\boldsymbol{a}_t$ is executed.

- Reward:

The reward of an action is a value that is generated by an environment after the action is executed in order to give feedback on the action to help an agent to perform better. The value reflects the effect of the action on the environment. For the considered system, the immediate reward of an action for $task\_k$ in time slot *t* is defined as

$$r_t^k = \begin{cases} +2, & \text{if } task\_k \text{ is completed} \\ -1, & \text{if } task\_k \text{ is not completed} \\ 0, & \text{otherwise} \end{cases}. \tag{15}$$

The immediate reward of the system in time slot *t* is defined as the sum of the immediate rewards of actions for all tasks in the system in time slot *t*, i.e.,

$$r_t = \sum_{k \in \mathcal{K}} r_t^k, \tag{16}$$

where r_t denotes the immediate reward of the system in time slot *t*. Obviously, the immediate reward of the system in time slot *t* has a positive correlation with the number of computation tasks successfully processed in time slot *t*.

- Policy

A policy is a mapping relationship between an observed state and an action. An agent's policy determines what action the agent will execute in response to a given observation. For the considered system, a UAV node's policy is to choose an action from the action space of the UAV based on the UAV's observation, which is called a local policy. The local policies of all UAV nodes in the system constitute a joint policy.

(2) Problem Transformation

According to [18], the optimization problem formulated in Eq. (14) is an approximately infinite continuing problem because the system continuously generates tasks and makes decisions on the tasks in a long period of time. It is not like an episodic problem, in which the interaction between an agent and an environment breaks naturally into episodes and each episode ends in a specific state. For such an approximately infinite continuing problem, an average reward with no discounting can be used to measure the quality of a policy [18], where no discounting means that no discount rate is put on a subsequent reward, i.e., a reward obtained subsequently has the same effect on the system as that obtained currently.

Let $r(\pi)$ denote the expected average reward of the system with a joint policy π, which is defined as

$$r(\pi) = \mathbb{E}[\lim_{T\to\infty} \frac{1}{T}\sum_{t=1}^{T}[r_t \mid a_{0:t-1} \sim \pi]], \quad (17)$$

where $a_{0:t-1}$ denotes a set of actions that the UAV nodes in the system execute from time slot 0 to time slot t-1 with the policy π. The expected average reward of the system with the policy π is the expectation of the average reward of the system in each time slot with the policy π and the state transition probability function $\mathcal{P}$.

Based on the definitions of the reward function and the expected average reward, the optimization problem P1 is transformed into an average-reward maximization problem with the objective to find an optimal joint policy π^* such that the expected average reward of the system in each time slot with the policy is maximized, i.e.,

P2:

$$\text{objective} \quad \pi^* = \arg\max_{\pi} r(\pi) = \arg\max_{\pi} \mathbb{E}[\lim_{T\to\infty} \frac{1}{T}\sum_{t=1}^{T} r_t]. \quad (18a)$$

$$\text{subject to} \quad \boldsymbol{a}_t^n \in \mathcal{A}_t^n, \forall \boldsymbol{a}_t^n \sim \pi, \quad (18b)$$

where $\mathcal{A}_t^n$ denotes the action space of UAV node U_n in time slot t and (18b) specifies that each action with the policy π must be selected from the action space of UAV node U_n in time slot t. Since the state of the system in each time slot is stochastic, the actual objective of solving problem P1 is to maximize the *expected* average number of computation tasks successfully processed in the system in each time slot. Obviously, maximizing the expected average reward of the system (i.e., the objective of P2) is equivalent to maximizing the expected average number of computation tasks successfully processed in the system in each time slot (i.e., the objective of P1). Therefore, problem P2 is equivalent to problem P1.

B. MARL-based Task Scheduling Algorithm

The MARL-based task scheduling algorithm is intended to solve problem P2, i.e., to find an optimal joint policy for each UAV to make scheduling decisions for each task arriving in a time slot such that the expected average reward of the system in each time slot with the policy is maximized. To achieve the objective, the algorithm introduces an MARL method and specifically uses QMIX [19], a typical CTDE method with a value decomposition approach, in its neural network training framework. The overall task scheduling process of the algorithm consists of two components: a neural network training process, and a task scheduling execution process. Next, we describe the neural network training framework, neural network training process, and task scheduling execution process in more details.

(1) Neural network training framework

In MARL, each agent uses a neural network to make scheduling decisions for computation tasks. The training effect of a neural network largely depends on the training framework of the neural networks. In general, MARL methods can be classified into three categories according to their training methods: fully centralized methods, fully decentralized methods, and centralized training and decentralized execution methods. In a fully centralized method, a multi-agent problem is reduced to a single-agent problem with multiple action entities. Training is performed by a central controller and a decision on a joint action is made by the central controller as well. In a fully decentralized method, e.g., independent Q-learning (IQL), agents are trained as independent individuals and make decisions only based on local observations. The former has a poor scalability and is impractical as global observations are needed for making each decision while the latter has the difficulty to deal with the non-stationarity of a multi-agent environment. A centralized training and decentralized execution (CTDE) method is a tradeoff between the two methods. In the CTDE method, each agent executes distributed actions in accordance with its individual policy and local observations while the policies of all agents are trained by a central controller. The CTDE method has a better scalability than a fully centralized method and can improve the non-stationarity of a multi-agent environment to a certain extent. To take these advantages, we use the CTDE method with a value decomposition approach in the proposed task scheduling algorithm. To implement this, we assume that one of the UAV nodes in the system is equipped with additional computation resources and serves as a central controller, which regularly collects observation-action history of other agents, trains the neural networks of all agents and updates the parameters of the neural networks, and then returns the updated parameters of the neural networks to each agent. In this case, the training of all agents' policies is performed online to adapt to an unknown environment.

Value-Decomposition Network (VDN) is a typical CTDE method with a value decomposition approach [20]. In VDN, the local Q-value functions of all agents are summed directly as a joint Q-value function, i.e.,

$$Q_{tot}((\boldsymbol{\tau}_t^1, \boldsymbol{\tau}_t^2, \ldots, \boldsymbol{\tau}_t^N), (\boldsymbol{a}_t^1, \boldsymbol{a}_t^2, \ldots, \boldsymbol{a}_t^N)) = \sum_{n=1}^{N} Q_n(\boldsymbol{\tau}_t^n, \boldsymbol{a}_t^n), \quad (19)$$

where Q_{tot} denotes the joint Q-value function for all agents and Q_n denotes the local Q-value function of agent n. As an extension

of VDN, QMIX uses a mixing network to approximately obtain the joint Q-value function Q_{tot} by performing a nonlinear combination operation on all agents' local Q-value functions. According to [19], the joint Q-value function can be expressed as

$$Q_{tot}(\boldsymbol{Q}) = f_2(\boldsymbol{W}_2 f_1(\boldsymbol{W}_1 \boldsymbol{Q})), \tag{20}$$

where $\boldsymbol{Q} = [Q_1, Q_2, \ldots, Q_N]$, $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$ are combination weights generated by a hypernetwork in the mixing network, f_1 is an nonlinear function, and f_2 is a linear function. In QMIX, the global state information on the environment is input into the hypernetwork to assist centralized training, enabling QMIX to deal with a more complicated environment. For the reason that the global state of the environment is only partly observable to each agent, the combination of local observations of all agents is used as the global state approximately, i.e., $s_t \approx \{o_t^n\}_{n=1}^N$. Moreover, Q_{tot} should have the same monotonicity as that of the local Q_n. This means that a globally optimal joint action should be equivalent to a set of all agents' locally optimal actions, i.e.,

$$\arg\max_{\boldsymbol{a}_t} Q_{tot}(\boldsymbol{\tau}_t, \boldsymbol{a}_t) = \begin{pmatrix} \arg\max_{\boldsymbol{a}_t^1} Q_1(\boldsymbol{\tau}_t^1, \boldsymbol{a}_t^1) \\ \vdots \\ \arg\max_{\boldsymbol{a}_t^N} Q_N(\boldsymbol{\tau}_t^N, \boldsymbol{a}_t^N) \end{pmatrix}. \tag{21}$$

The monotonicity of Q_{tot} and that of each Q_n are enforced by the following constraint:

$$\frac{\partial Q_{tot}}{\partial Q_n} \geq 0, \forall n \in \{1, \ldots, N\}. \tag{22}$$

To satisfy constraint (22), the combination weights $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$ generated by the hypernetwork are restricted to being non-negative. Thus, each agent only needs to follow the local policy of maximizing its individual Q-value function. Therefore, we introduce QMIX to train the neural networks of all agents in the proposed algorithm. The overall neural network training framework used in the proposed algorithm is composed of a mixing network and a set of agent networks, which is shown in Fig. 2.

An agent network (i.e., the neural network of an agent) consists of a deep Q-network (DQN) and an RNN, in which FC means a fully connected layer and GRU means a gate recurrent unit, as shown in Fig. 3. The DQN is used to fit the local Q-value function of the agent and generate or output the Q-values of actions for each task. Considering that the number of tasks arriving in each time slot is variable, one step during an agent's independent decision process (i.e., a time slot in the system) can be divided into several sub-steps. The number of sub-steps in one step is equal to the number of tasks for which the agent needs to make decisions. In each sub-step, the observation on the first task in the arrival task queue $\boldsymbol{q}_{n,t}^a$ is input to the DQN with the current local observation on the agent's local task queues, and then the action with the highest Q-value among all Q-values output by the DQN is selected as the optimal action for the task. After the action is selected, it will be executed by the agent and the task will be moved to either the computation task queue $\boldsymbol{q}_{n,t}^c$ or the offloading task queue $\boldsymbol{q}_{n,t}^o$ from $\boldsymbol{q}_{n,t}^a$, depending on the action. Then, the observation on the agent's local task queues will be updated. The updated observation is called a post observation and will be used as the local observation in the next sub-step. The agent can obtain its Q-value in one step by summing the Q-values in all sub-steps of the step.

However, a DQN can only make use of an observation in the current step but cannot quickly respond to a more dynamic environment in multiple steps. For the partial observability of POMDP, the errors of local Q-values generated by a DQN will increase, i.e., $Q(o,a) \neq Q(s,a)$. An RNN has a short-term memory ability and its hidden layer can keep part of input information, enabling an RNN to conjecture the global state from sequential observations in multiple steps. Thus, we introduce RNNs in the proposed algorithm to deal with the partial observability of POMDP [21] and assist the DQNs to select actions. In the considered system, each agent inputs its observation (apart from the local observation on its local task queues) into its RNN. Then, the RNN will output an auxiliary variable z_t^n for the DQN, which will be input into the DQN with the observations on the tasks in each sub-step of the current step.

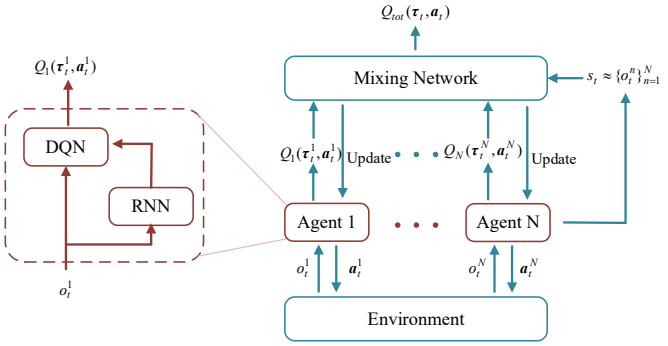

Fig. 2. Training framework of the neural networks.

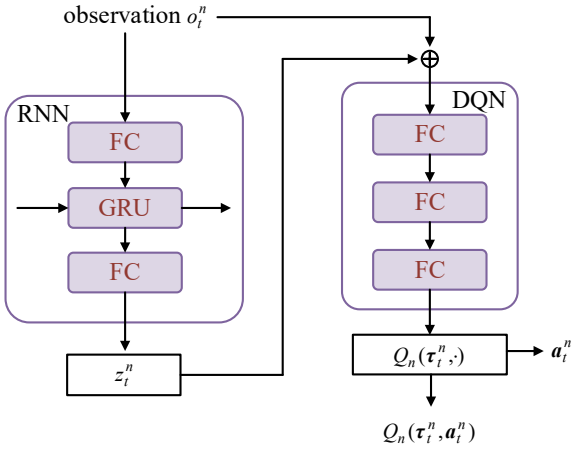

Fig. 3. Neural network structure of each agent.

(2) Neural network training process

The neural network training process is intended to train the mixing network of the central controller and the neural networks of all agents in order to learn an optimal joint policy for all agents. To this end, Experience Replay and Target Network are used to improve the training effect. The Experience Replay is used to improve data diversity of the observation-action history used for training and improve training efficiency. In each step during the

task scheduling execution process, each agent takes actions according to the current local policy, and its observation-action history will be collected by the central controller and is stored into a replay buffer as an experience. During the neural network training process, the central controller will sample a batch of experiences from the replay buffer to train all neural networks, i.e., update the parameters of all neural networks. Then, the central controller will return the updated parameters to each agent and each agent will update its local policy accordingly. The process is iterated continuously to adjust the policies to the environment.

The Target Network is used to alleviate overestimation of using a single DQN. The DQN of each agent includes an evaluation network and a target network. The evaluation network is used to make task scheduling decisions while the target network is used to assist the training of the evaluation network. During the training process, the parameters of the mixing network and all agents' DQNs are updated by performing gradient backpropagation to minimize a global temporal-difference (TD) error, which is defined as

$$\delta = r - r(\pi) + \max_{\boldsymbol{a}'} Q_{tot}(\boldsymbol{\tau}', \boldsymbol{a}'; \boldsymbol{\theta}') - Q_{tot}(\boldsymbol{\tau}, \boldsymbol{a}; \boldsymbol{\theta}), \quad (23)$$

where r denotes the immediate reward of the system in a step with an experience among a batch of sampled experiences, $\boldsymbol{\tau}/\boldsymbol{\tau}'$ denotes an observation-action history in the step/the next step, $\boldsymbol{a}/\boldsymbol{a}'$ denotes the joint action in the step/the next step, $\boldsymbol{\theta}$ and $\boldsymbol{\theta}'$ denote the parameters of the evaluation networks and target networks, respectively, and $Q_{tot}(\boldsymbol{\tau}, \boldsymbol{a}; \boldsymbol{\theta})$ and $\max_{\boldsymbol{a}'} Q_{tot}(\boldsymbol{\tau}', \boldsymbol{a}'; \boldsymbol{\theta}')$ denote the joint Q-values obtained from the local Q-values generated by the evaluation networks and those obtained from the local Q-values generated by the target networks, respectively. The parameters of all agents' RNNs are updated after the parameters of the DQNs are updated.

(3) Task scheduling execution process

In the task scheduling execution process, each UAV (or agent) makes task scheduling decisions based on its local observation according to an ε -exploration strategy and the newest local policy trained by the central controller. Specifically, in each time slot, each UAV needs to take actions for the computation tasks in its arrival task queue in order of arrival time. For each of the tasks, the agent inputs its observation into its neural network (i.e., its RNN and DQN) to obtain a batch of Q-values of actions and select an action with the highest Q-value for the task. Then, the action will be executed and the task will be moved to either the computation task queue or the offloading task queue from the arrival task queue depending on the action. After actions for all the tasks are selected and executed, task scheduling for the current time slot is completed and task scheduling for the next time slot will start.

The overall task scheduling process of the proposed MTS algorithm can be described by the given pseudo-codes.

(4) Computational complexity

Let N_D^i denote the number of neurons at the i-th layer of a DQN, L_D denote the number of hidden layers in a DQN, and S denote the batch size of sampled experiences in one training of all neural networks in the algorithm. Thus, the computational complexity for training a DQN network can be given by $\mathcal{C}_D = \mathcal{O}(S \cdot \sum_{i=0}^{L_D} (N_D^i \cdot N_D^{i+1}))$. Similarly, the computational complexity for training an RNN network can be given by $\mathcal{C}_R = \mathcal{O}(S \cdot \sum_{j=0}^{L_R} (N_R^j \cdot N_R^{j+1}))$, where N_R^j denotes the number of neurons at the j-th layer of the RNN and L_R denotes the number of hidden layers in the RNN. The computational complexity for training a mixing network can be given by $\mathcal{C}_M = \mathcal{O}(S \cdot \sum_{k=0}^{L_M} (N_M^k \cdot N_M^{k+1}))$, where N_M^k denotes the number of neurons at the kth layer of the mixing network and L_M denotes the number of hidden layers in the mixing network. Considering that there are N agent networks and one mixing network in the training framework of the MTS algorithm, the overall computational complexity of the MTS algorithm is given by $N \cdot (\mathcal{C}_D + \mathcal{C}_R) + \mathcal{C}_M$.

MTS Algorithm

Initialize the environment;
Initialize hyper parameters for learning and an average-reward estimate $r(\pi)$;
Initialize network parameters $\boldsymbol{\theta}$ of the mixing network and DQNs, network parameters $\boldsymbol{\omega}$ of RNNs and a replay buffer;
Set the parameters of target networks $\boldsymbol{\theta}' = \boldsymbol{\theta}$;
for $t = 1, \ldots, T$ **do**
  for $n = 1, \ldots, N$ **do**
    Obtain observation o_t^n;
    Take o_t^n as an input to the RNN network and obtain an observation variable z_t^n;
    Divide step t into sub-steps according to $\boldsymbol{q}_{n,t}^a$;
    for each sub-step **do**
      Combine z_t^n and o_t^n as an input to the DQN network;
      Choose an action according to the outputs of DQN and the ϵ-exploration strategy;
      Update observation o_t^n;
    end for
    All actions make up an agent action $\boldsymbol{a}_t^n$;
  end for
  Execute a joint action $\boldsymbol{a}_t = (\boldsymbol{a}_t^1, \boldsymbol{a}_t^2, \ldots, \boldsymbol{a}_t^N)$;
  Observe $\boldsymbol{o}_{t+1} = \{o_{t+1}^n\}_{n=1}^N$ and obtain reward r_t
  Store $\{(o_t^n, z_t^n, \boldsymbol{a}_t^n, r_t, o_{t+1}^n)\}_{n=1}^N$ into the replay buffer;
  if t mod train_cycle == 0 **then** ▷ *train_cycle* denotes the training cycle length of the neural networks (i.e., the number of steps between two trainings).
    Sample a batch from the replay buffer;
    Calculate the TD-error $\delta = r - r(\pi) + Q_{tot}(\boldsymbol{\tau}', \boldsymbol{a}'; \boldsymbol{\theta}') - Q_{tot}(\boldsymbol{\tau}, \boldsymbol{a}; \boldsymbol{\theta})$;
    Update $r(\pi)$;
    Perform backpropagation and update the parameters $\boldsymbol{\theta}$ of the mixing network and DQNs ;
    Update the parameters $\boldsymbol{\omega}$ of RNNs ;
    if t mod target_update_cycle == 0 **then** ▷ *target_update_cycle* denotes the updating cycle length of the target networks (i.e., the number of steps between two updates).
      Update the parameters $\boldsymbol{\theta}'$ of the target networks;
    end if
  end if
end for

V. Performance Evaluation

In this section, we evaluate the performance of the proposed MTS algorithm through simulation results.

A. Simulation Setup

The simulation experiments were conducted using Python 3.8 with Pytorch 1.12.1. In the experiments, we consider a square area of 500 m by 500 m, which consists of 5 randomly distributed service areas with one UAV deployed over the center of each service area, respectively. The radius of each service area is set to 50 m. Ground users are randomly generated within the service areas. The other main parameters used in the experiments are listed in Table I.

In the proposed MTS algorithm, the neural network training framework is composed of a mixing network and a set of agent networks. Each DQN in the agent networks has two hidden layers of 256 neurons. Each RNN in the agent networks has one recurrent layer consisting of a GRU with 256-dimension hidden states. The mixing network has a hidden layer of 128 neurons. An Adam optimizer is used for neural network training with different learning rates. The learning rate of all agent networks is 0.001 and that of the mixing network is 0.0005. The batch size of experiences in one training is set to 32, each containing 60 continuous steps. The DNNs and RNNs are trained every 30 steps and 3×30 steps, respectively. In addition, the exploration rate ε exponentially decays from 1 to 0.2 with a decay coefficient 1500.

Table I Main Parameter Settings

Parameter	Value
N	5
B^G	1 MHz
P^G	0.5 W
$f^m, f^{nn'}$	2 GHz
B^U	2 MHz
P^U	2 W
d^k	50~100 Kb
c^k	800 cycle/bit
$dis^{\max}$	200 m
h^N	50 m
p_{gen}	0.8
f^{CPU}	3*10^8 cycle/s
N_0	-100 dBm
$(\eta_{LoS}, \eta_{NLoS}, a, b)$	(1, 20, 9.61, 0.16) [17]
$\tau^k_{\max}$	7~12 s
Δt	1 s

In the performance evaluation, we use the immediate reward of the system in one step, the average number of tasks successfully processed in the system in one time slot, the success rate of the system, and the resource utilization of the system in one time slot as the performance metrics. The success rate is defined as the ratio of the number of tasks successfully processed to the total number of generated tasks in the system. The resource utilization in one time slot is defined as the ratio of the total amount of computation resources (i.e., CPU frequency) used in one time slot to the total amount of computing resources of all UAV nodes in the system. Moreover, we use the following algorithms as benchmark algorithms in the evaluation:

1) No Cooperation (No-Coop): a simple algorithm in which each UAV node only provides computing services for its local ground users.
2) VDN-based (VDN): a VDN-based task scheduling algorithm in which the neural network of the central controller is a summing network and each agent network is similar to that in the proposed algorithm.
3) IQL-based (IQL): an IQL-based task scheduling algorithm which uses an independent Q-learning method to learn an optimal policy in a fully decentralized manner.

B. Simulation Results

We first evaluate the convergence performance with the MTS algorithm for different training parameters to determine optimal values for the training parameters.

Fig. 4 shows the convergence performance of the MTS algorithm for different training parameters, including the learning rate of all agent networks (α), the number of neurons at each layer of an agent network (N_{nero}), the batch size (S), and the training interval (ΔT). Fig. 4(a) shows the convergence performance for different values of α. It is seen that for α =0.001, the immediate reward of the system converges more smoothly and reaches the largest value compared to those for α =0.0005 and α =0.002. Fig. 4(b) shows the convergence performance for different values of N_{nero}. It is seen that the immediate reward of the system for N_{nero} =256 converges more smoothly and reaches a larger value compared to those for N_{nero} =128 and N_{nero} =512. Fig. 4(c) shows the convergence performance for different values of S. It is seen that for S=32, the immediate reward of the system converges more smoothly and reaches the largest value compared to those for S=16 and S=64. Fig. 4(d) shows the convergence performance for different values of ΔT. It is seen that for ΔT =30, the immediate reward of the system converges more smoothly and reaches the largest value compared to those for ΔT =15 and ΔT =60. Therefore, we set α =0.001, N_{nero} =256, S=32, and ΔT =30 in all the following simulation experiments.

Next we investigate the performance of the proposed algorithm under three different load conditions: 1) over-but-balanced load, 2) unbalanced load, and 3) varying load, through simulation results.

1) Performance under an over-but-balanced load condition

In this experiment, the number of ground users distributed in each service area was set to 10. In this case, all UAV nodes in the system are unable to complete all tasks generated by the ground users in their service areas.

Fig. 5 shows the performances with the MTS algorithm and the No-Coop algorithm under the overloaded condition, respectively. It is observed that all the four performances with the MTS algorithm converge quickly while those with the No-Coop algorithm keep stable values after initialization. After

This article has been accepted for publication in IEEE Transactions on Vehicular Technology. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TVT.2025.3635005

convergence, the two algorithms have almost the same performances with the resource utilization in one time slot equal to 1 and the MTS algorithm does not show better performance than the No-Coop algorithm. This is because in this case the system is under an overloaded condition and thus task offloading between different UAV nodes cannot bring any improvement to the performances.

2) Performance under an unbalanced-load condition

In this experiment, the number of ground users distributed in each service area was set to [2, 15, 3, 8, 20]. In this case, some UAVs are overloaded while the others are non-overloaded.

Fig. 6 shows the performances with the MTS algorithm and the No-Coop algorithm. It is observed that all the four performances with the MTS algorithm can converge, while those with the No-Coop algorithm keep stable values after initialization. After convergence, all the four performances with the MTS algorithm are better than those with the No-Coop algorithm. This is because with the MTS algorithm all UAVs in the system provide computation service in a cooperative manner and an overloaded UAV node can offload its tasks to a non-overloaded UAV node according to an optimal task scheduling policy learnt by the algorithm.

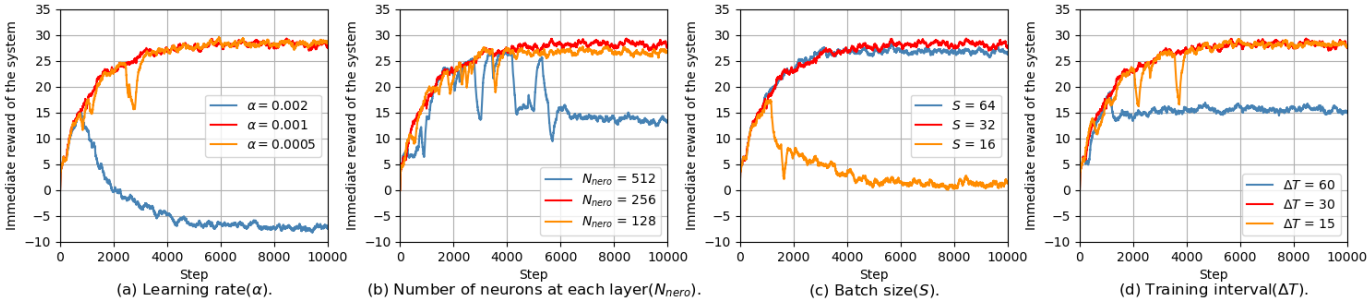

Fig. 4. Convergence performance for different training parameters.

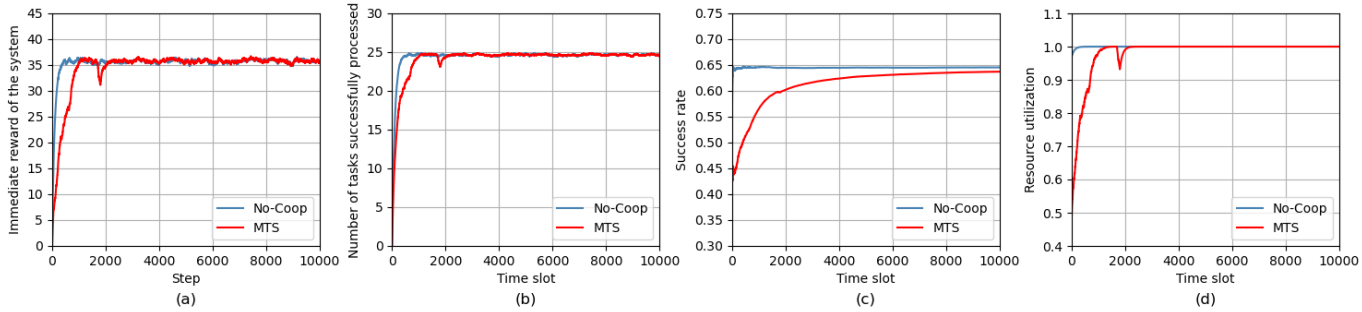

Fig. 5. Performances under an overload condition.

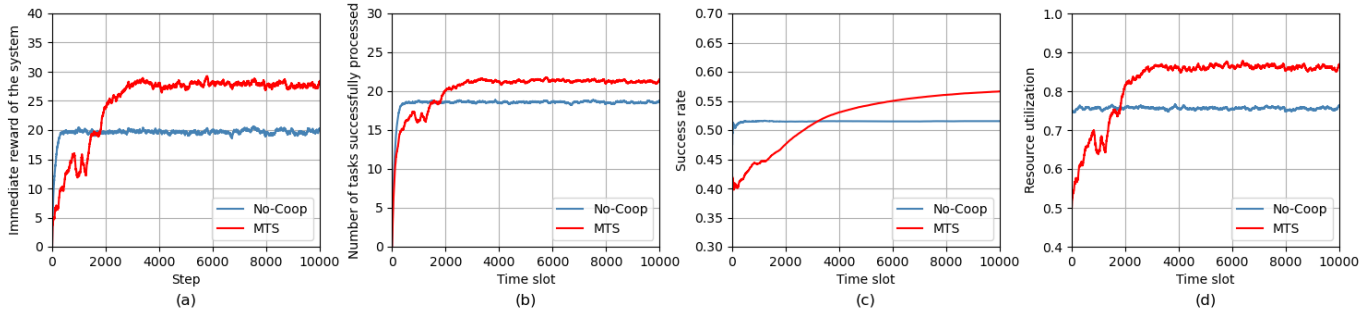

Fig. 6. Performances under an unbalanced-load condition.

3) Performance under a varying-load condition

In this experiment, the number of ground users in each service area was set to 20. The generation rate of a ground user in each service area varies among [0, 0.4, 0.8] periodically, in every 30 time slots, following a pattern shown in Fig. 7. In this case, the load condition of each UAV node varies between an overloaded condition and a non-overloaded condition.

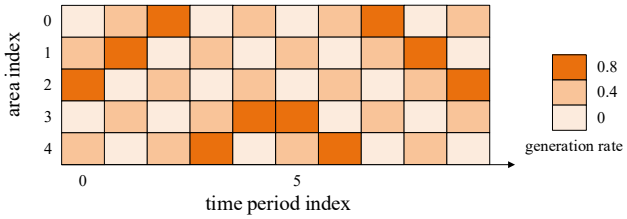

Fig. 7. Illustration of a periodically-varying load pattern.

This article has been accepted for publication in IEEE Transactions on Vehicular Technology. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TVT.2025.3635005

Fig. 8 shows the performances with the MTS algorithm and the other three benchmark algorithms over time, respectively. It is observed that the MTS algorithm converges more quickly and smoothly than the VDN and IQL algorithms. This is because with the MTS algorithm the policies of the UAV nodes are trained in a centralized manner, while with the IQL algorithm the policies of the UAV nodes are trained independently in a decentralized manner. Moreover, with the MTS algorithm the policies of the UAV nodes are trained using a mixing network, while with the VDN algorithm the policies of the UAV nodes are trained using a simple summing network. On the other hand, it is seen that after convergence the MTS algorithm outperforms all the other three algorithms.

Fig. 5, Fig. 6 and Fig. 8 show that the MTS algorithm converges quickly and smoothly under all the three conditions, and Fig. 8 also shows that the MTS performs better than other learning-based benchmark algorithms before convergence. This is important for an online learning algorithm. Next, we investigate the impacts of the CPU frequency of a UAV, the number of ground users in each service area, and the maximum tolerant delay of a task on the performances with the MTS algorithm, VDN, IQL, and No-Coop algorithms, respectively.

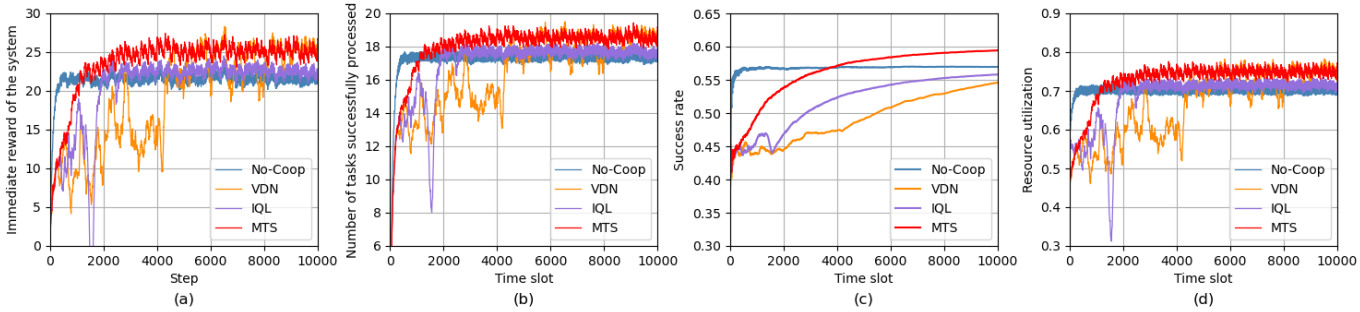

Fig. 8. Performances under a periodically-varying load condition.

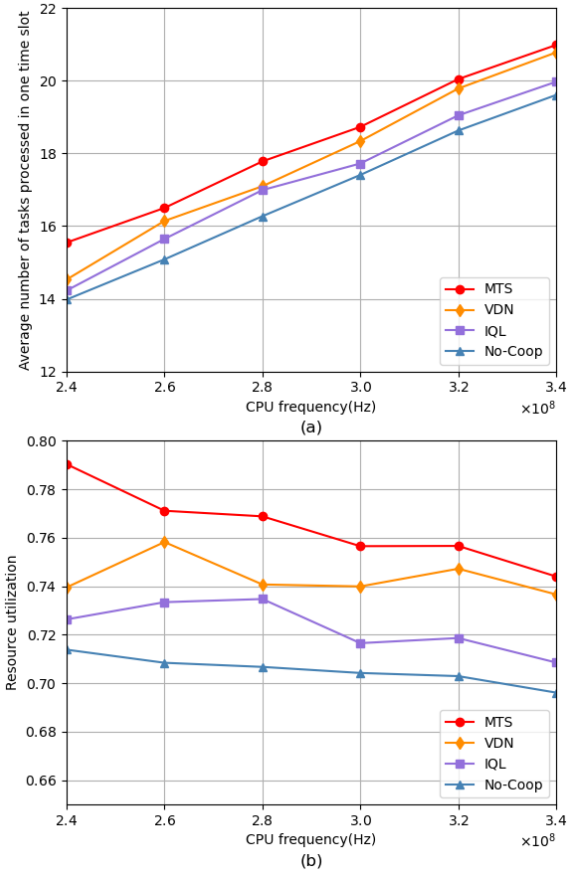

Fig. 9. Performances vs CPU frequency.

Fig. 9(a) shows the average numbers of tasks successfully processed in one time slot vs the CPU frequency of a UAV with the MTS algorithm and the three benchmark algorithms, respectively. It is observed that as the CPU frequency increases, the average numbers of tasks successfully processed with the four algorithms all gradually increase. The reason is that when the CPU frequency increases, the processing ability of each UAV becomes stronger, which results in a smaller computation delay for processing a task, no matter whether the task is processed locally or offloaded to a neighboring UAV for processing. Thus, more tasks can be completed in one time slot. On the other hand, it is seen that the average numbers of tasks successfully processed with the MTS, VDN and IQL algorithms are larger than that with the No-Coop algorithm. This is because with the MTS, VDN, or IQL algorithm all UAV nodes in the system operates in a cooperative manner and an overloaded UAV node can offload its tasks to a non-overloaded UAV node according to an optimal task scheduling policy learnt by the algorithm, while with the No-Coop algorithm a UAV node is not allowed to offload tasks to other nodes. Moreover, the average number of tasks successfully processed with the MTS algorithm is larger than that with the VDN algorithm. This is because the mixing network used in the MTS algorithm enables the agents to learn a better task scheduling policy than the policy learnt with the VDN algorithm. The average number of tasks successfully processed with the MTS algorithm is larger than that with the IQL algorithm as well. The reason is that the MTS algorithm uses a QMIX method to train the agents in a centralized manner and introduces RNNs to assist decision making, while with the IQL algorithm the agents are trained in a decentralized manner and use a single DQN to make task scheduling decisions in the system.

Fig. 9(b) shows the resource utilizations vs the CPU frequency of a UAV with the MTS algorithm and the three benchmark algorithms, respectively. It is observed that in general, as the CPU frequency increases, the resource utilization with the MTS algorithm and those with the three benchmark algorithms all tend to decrease. This is because in this case, the computation resources used by overloaded UAVs will increase while those used by non-overloaded UAVs will not increase. However, the increase in the total amount of computation resources used by both overloaded and non-overloaded UAVs is smaller than that in the total amount

This article has been accepted for publication in IEEE Transactions on Vehicular Technology. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TVT.2025.3635005

of computation resources of all UAVs, causing the decrease of the resource utilization. On the other hand, it is observed that the resource utilizations with the MTS, VDN and IQL algorithms are larger than that with the No-Coop algorithm. This is because with the MTS, VDN or IQL algorithms a UAV can offload its tasks to neighboring non-overloaded UAVs, which can make more sufficient use of the computation resources in the system. Moreover, the resource utilization with the MTS algorithm is larger than those with the VDN and IQL algorithms. This is because with the MTS algorithm the agents can learn a better task scheduling policy than those with the VDN and IQL algorithms.

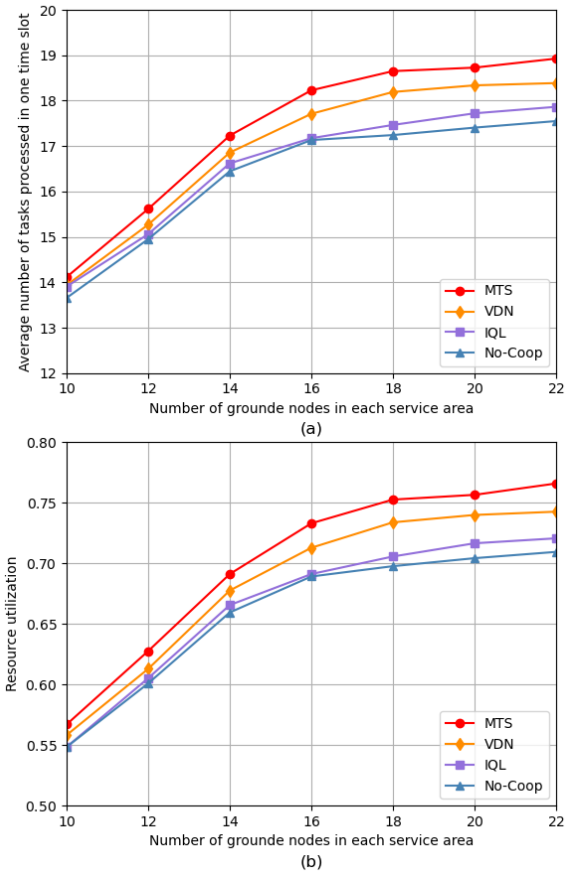

Fig. 10. Performances vs number of ground users in each service area.

Fig. 10(a) shows the average numbers of tasks successfully processed in one time slot vs the number of ground users in each service area with the MTS, VDN, IQL, and No-Coop algorithms, respectively. It is observed that as the number of ground users in each service area increases, the average numbers of tasks successfully processed with the four algorithms all gradually increase and tend to be stable. The reason is that when the number of ground users is small, a UAV is under a non-overload condition and thus can process more tasks when the number of ground users increases. However, when the number of ground users increases beyond a certain extent, most of the UAVs become overloaded, which makes the increase in the number of tasks successfully processed slow down. On the other hand, the average number of tasks successfully processed with the MTS algorithm is larger than that with the No-Coop algorithm. The average number of tasks successfully processed with the MTS algorithm is larger than those with the VDN and IQL algorithms. The reasons can be explained similar to those in Fig. 9(a).

Fig. 10(b) shows the resource utilizations vs the number of ground users in each service area with the MTS, VDN, IQL and No-Coop algorithms, respectively. It is observed that as the number of ground users in each service area increases, the resource utilizations with the four algorithms all gradually increase. The reason is that in this case, the number of tasks generated by the ground users increases, which results in each UAV to use more computation resources to process the tasks.

Fig. 11(a) shows the average numbers of tasks successfully processed in one time slot vs the maximum tolerant delay of a task with the MTS, VDN, IQL, and No-Coop algorithms, respectively. Fig. 11(b) shows the resource utilizations vs the maximum tolerant delay of a task with the MTS, VDN, IQL, and No-Coop algorithms, respectively. It is observed that as the maximum tolerant delay increases, both the average numbers of tasks successfully processed and the resource utilizations of the system with all the four algorithms increase. This is because as the maximum tolerant delay increases, each task can wait a longer time for being processed. As a result, more tasks are completed and more computation resources are used to process the tasks. On the other hand, the number of tasks successfully processed with the MTS algorithm is larger than those with the other three algorithms. The resource utilization with the MTS algorithm is larger than those with the other three algorithms as well. The reasons can be explained similar to those in Fig. 9(a).

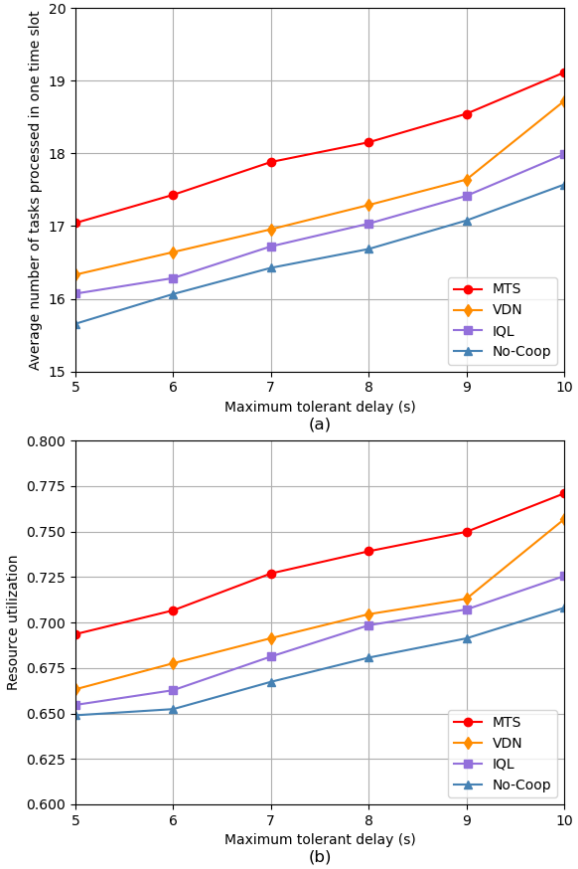

Fig. 11. Performances vs maximum tolerant delay.

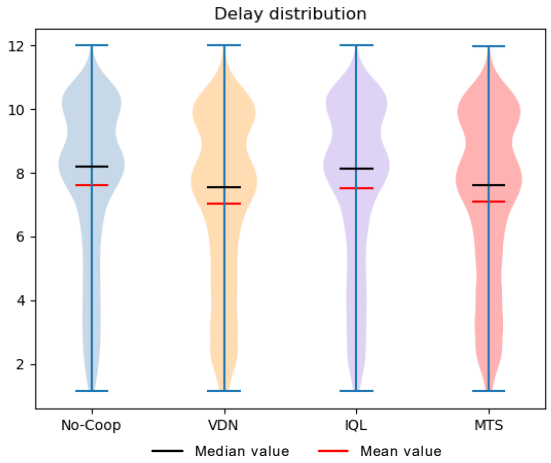

Fig. 12. The delay distribution of the tasks successfully processed.

Fig. 12 shows the delay distribution of the tasks successfully processed in the system with the MTS algorithm and the other three algorithms, respectively. Since the maximum tolerant delay of a computing task randomly ranges from 7s to 12s, the delay of a computing task successfully processed in the system is lower than 12s. It is seen that the mean value of the service delays with the MTS algorithm is close to that with the VDN algorithm, and is smaller than those with the No-Coop and IQL algorithms. Similarly, the median value of the service delays with the MTS algorithm is close to that with the VDN algorithm, and is smaller than those with the No-Coop and IQL algorithms.

VI. Conclusions

In this paper, we investigated the task scheduling problem in a multi-UAV edge computing system using an RL method. We formulated the problem as an integer nonlinear programming problem with an objective to maximize the average number of computation tasks successfully processed in each time slot by optimizing the task scheduling decision for each task generated in the system. To solve the formulated problem using an RL method, we first reformulated the problem as a Dec-POMDP and further transformed the problem into an average-reward maximization problem, and then proposed the MTS algorithm to solve the transformed problem. In the proposed MTS algorithm, a CTDE method QMIX is introduced into the neural network training framework, in which the local policies of all agents are jointly trained by a central controller in a centralized manner and each agent executes actions independently according to its local observation and local policy in a decentralized manner. Moreover, RNNs are also introduced to deal with the partial observability of POMDP. Using the proposed MTS algorithm, each UAV is able to make optimal scheduling decisions independently for efficient cooperative task offloading between UAVs and thus provides better computing services for ground users. The simulation results show that the proposed MTS algorithm can achieve a larger average number of tasks successfully processed in the system compared with the VDN, IQL, and No-Coop algorithms.

References

[1] L. Gupta, R. Jain, and G. Vaszkun, "Survey of Important Issues in UAV Communication Networks," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1123–1152, 2016.

[2] Z. Lin, M. Lin, T. de Cola, J.-B. Wang, W.-P. Zhu, and J. Cheng, 'Supporting IoT with Rate-Splitting Multiple Access in Satellite and Aerial-Integrated Networks', *IEEE Internet Things J.*, vol. 8, no. 14, pp. 11123–11134, Jul. 2021.

[3] X. Xia, S. M. M. Fattah, and M. A. Babar, "A Survey on UAV-Enabled Edge Computing: Resource Management Perspective," *ACM Comput. Surv.*, vol. 56, no. 3, pp. 1–36, Mar. 2024.

[4] Y. Sun et al., 'Multi-Functional RIS-Assisted Semantic Anti-Jamming Communication and Computing in Integrated Aerial-Ground Networks', *IEEE J. Sel. Areas Commun.*, vol. 42, no. 12, pp. 3597–3617, Dec. 2024.

[5] H. Guo, Y. Wang, J. Liu and C. Liu, "Multi-UAV Cooperative Task Offloading and Resource Allocation in 5G Advanced and Beyond," *IEEE Trans. Wireless Commun.*, vol. 23, no. 1, pp. 347-359, Jan. 2024.

[6] Q. Luan, H. Cui, L. Zhang, and Z. Lv, "A Hierarchical Hybrid Subtask Scheduling Algorithm in UAV-Assisted MEC Emergency Network," *IEEE Internet Things J.*, vol. 9, no. 14, pp. 12737–12753, Jul. 2022.

[7] Z. Qin et al., "AoI-Aware Scheduling for Air-Ground Collaborative Mobile Edge Computing," *IEEE Trans. Wireless Commun.*, vol. 22, no. 5, pp. 2989–3005, May 2023.

[8] S. Li et al., "Joint Computation Offloading and Multidimensional Resource Allocation in Air–Ground Integrated Vehicular Edge Computing Network," *IEEE Internet Things J.*, vol. 11, no. 20, pp. 32687-32700, Oct., 2024.

[9] Z. Kuang, H. Wang, J. Li, and F. Hou, "Utility-Aware UAV Deployment and Task Offloading in Multi-UAV Edge Computing Networks," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 14755–14770, Apr. 2024.

[10] Y. K. Tun, T. N. Dang, K. Kim, M. Alsenwi, W. Saad, and C. S. Hong, "Collaboration in the Sky: A Distributed Framework for Task Offloading and Resource Allocation in Multi-Access Edge Computing," *IEEE Internet Things J.*, vol. 9, no. 23, pp. 24221–24235, Dec. 2022.

[11] X. Chen et al., "Information Freshness-Aware Task Offloading in Air-Ground Integrated Edge Computing Systems," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 1, pp. 243-258, Jan. 2022.

[12] H. Kang, X. Chang, J. Mišić, V. B. Mišić, J. Fan, and Y. Liu, "Cooperative UAV Resource Allocation and Task Offloading in Hierarchical Aerial Computing Systems: A MAPPO-Based Approach," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10497–10509, Jun. 2023.

[13] H. Yu, S. Leng, and F. Wu, "Joint Cooperative Computation Offloading and Trajectory Optimization in Heterogeneous UAV-Swarm-Enabled Aerial Edge Computing Networks," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 17700-17711, May. 2024.

[14] Q. Luo, T. H. Luan, W. Shi and P. Fan, "Deep Reinforcement Learning Based Computation Offloading and Trajectory Planning for Multi-UAV Cooperative Target Search," IEEE J. Sel. Areas Commun., vol. 41, no. 2, pp. 504-520, Feb. 2023.

[15] W. Xu, T. Zhang, X. Mu, Y. Liu, and Y. Wang, "Trajectory Planning and Resource Allocation for Multi-UAV Cooperative Computation," *IEEE Trans. Commun.*, vol. 72, no. 7, pp. 4305–4318, Jul. 2024.

[16] X. Li, Y. Qin, J. Huo and W. Huangfu, "Computation Offloading and Trajectory Planning of Multi-UAV-Enabled MEC: A Knowledge-Assisted Multiagent Reinforcement Learning Approach," *IEEE Trans. Veh. Technol.*, vol. 73, no. 5, pp. 7077-7088, May. 2024.

[17] A. Al-Hourani, S. Kandeepan, and S. Lardner, "Optimal LAP altitude for maximum coverage," *IEEE Wireless Commun. Lett.*, vol. 3, no. 6, pp. 569–572, 2014.

[18] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction. MIT press, 2018.

[19] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. ICML*, 2018, pp. 4295–4304.

[20] P. Sunehag et al., "Value-decomposition networks for cooperative multiagent learning based on team reward," in *Proc. AAMAS*, 2018, pp. 2085–2087.

[21] M. J. Hausknecht and P. Stone, "Deep recurrent Q-learning for partially observable MDPs," in *Proc. AAAI Fall Symposia*, 2015, pp. 29–33.

Biography: Luyinru Yang received the Bachelor's degree in Information Engineering from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2019. She is currently pursuing the Ph.D. degree with the School of Information Science and Engineering, Southeast University, Nanjing, China. Her research interests include edge computing, unmanned aerial vehicles and reinforcement learning.

Jun Zheng received a Ph. D. degree in Electrical and Electronic Engineering from The University of Hong Kong, Hong Kong, in 2000. He is currently a Full Professor with the School of Information Science and Engineering at Southeast University (SEU), China, and also a member with the Purple Mountain Laboratories, China. Before joining SEU in 2008, he was with the School of Information Technology and Engineering, University of Ottawa, Canada. He has co-authored (first author) two books published by Wiley-IEEE Press, and has authored or co-authored over 200 technical papers in refereed journals and peer-reviewed conference proceedings. His current research interests include mobile communication networks, vehicular ad hoc networks, and UAV-connected networks, focused on network architectures and protocols. He was a co-recipient of the Best Paper Awards at IEEE ICC 2014 and WCSP 2018. He is now an editorial board member of several international journals, including IEEE Transactions on Vehicular Technology and IEEE Transactions on Cognitive Communications and Networking. He has co-edited fourteen special issues for different refereed journals and magazines, including IEEE Communications Magazine, IEEE Network, and IEEE Journal on Selected Areas in Communications, all as Lead Guest Editor. He has served as the founding General Chair of AdHocNets 2009, General Chair of AccessNets 2007, AdHocNets 2018/2019, ICNC 2024, and TPC or Symposium Co-Chair for a number of international conferences and symposia, including IEEE ICC 2009/2011/2015/2021 and GLOBECOM 2008/2010/2012/2018/2019. He has also served as a TPC member for a number of international conferences and symposiums. He is a senior member of the IEEE, IEEE Communications Society, and IEEE Vehicular Technology Society. He serves as Chair of IEEE Vehicular Technology Society Nanjing Chapter, and was Chair of IEEE ComSoc Communications Switching and Routing Technical Committee. He is a recipient of 2021 IEEE ComSoc Communications Software Technical Committee "Technical Achievement Award" and 2023 IEEE ComSoc Communications Switching & Routing Technical Committee "Distinguished Technical Achievement Award".

Yuan Zhang received the Bachelor degree and Ph.D. degree from Southeast University, Nanjing, China, in 1998 and 2004, respectively. Since 2005, he has been working with the National Mobile Communications Research Laboratory at Southeast University as a faculty member. His research interests are in the areas of wireless communications, networking, and computing. He has published over fifty technical papers in journals and conference proceedings. He is a co-receipient of two best paper awards at ICC 2014 and ICCC 2019.